Experiment No. 01	Setup data engineering environment: Docker, Python, Jupyter, Git
Date	

AIM

To set up a basic data engineering environment using Python, Jupyter Notebook, Docker, and Git for data processing, experimentation, and version control.

ALGORITHM

1. Install Python and verify its installation.
2. Install Jupyter Notebook using pip.
3. Install Docker Desktop and verify its installation.
4. Install Git and configure the username and email.
5. Create a project directory for data engineering.
6. Create and run a Python program to verify the environment.
7. Launch Jupyter Notebook to execute Python code interactively.
8. Use Git to track changes in the project.
9. Verify that all tools are working successfully

PROGRAM / COMMANDS

Step 1: Check Python Installation

python --version

If Python does not work on Windows:

py –version

Step 2: Check PIP Installation

pip --version

Step 3: Install and Verify Jupyter Notebook

pip install notebook

jupyter –version

Step 4: Launch Jupyter Notebook

jupyter notebook

Execute the following code in a new Python notebook:

print("Data Engineering Environment Ready")

Step 5: Check Docker Installation

docker --version

docker run hello-world

Step 6: Check Git Installation

git --version

Step 7: Configure and Verify Git

git config --global user.name "Your Name"

git config --global user.email "your@email.com"

git config –list

OUTPUT

Python Output

Python 3.12.6

PIP Output

pip 24.x.x from ... (python 3.12)

Jupyter Output

IPython 8.x.x

ipykernel 6.x.x

jupyter_client 8.x.x

jupyter_core 5.x.x

notebook 7.x.x

Jupyter Notebook Execution

Data Engineering Environment Ready

Docker Output

Docker version 27.x.x, build xxxxx

After running docker run hello-world:

Hello from Docker!

Your installation appears to be working correctly.

Git Output

git version 2.46.x

Git Configuration Output

user.name=Your Name

user.email=your@email.com

INFERENCE

The data engineering environment was configured using Python, Jupyter Notebook, Docker, and Git. Python executes programs, Jupyter supports interactive data analysis, Docker provides containerization, and Git manages project versions.

VIVA VOCE

Q1. What is Data Engineering?

Q2. What is Python used for in Data Engineering?

Q3. What is Jupyter Notebook?

Q4. What is Docker?

Q5. What is Git?

RESULT

Thus, the basic data engineering environment using Python, Jupyter Notebook, Docker, and Git was successfully set up and verified.

Experiment No. 02	Implement ETL Pipeline Using Python (Pandas) for CSV/JSON Data Processing
Date	

AIM

To implement an ETL (Extract, Transform, Load) pipeline using Python and Pandas to read CSV and JSON data, clean and transform the data, and save the processed data into an output file.

ALGORITHM

1. Start the program.
2. Import the Pandas library.
3. Create sample data and save it in CSV and JSON formats.
4. Extract the data by reading the CSV and JSON files.
5. Transform the data by removing duplicate records and handling missing values.
6. Load the cleaned data into a new CSV file.
7. Display the original and processed data.
8. Stop the program.

PROGRAM / COMMAND:

```
import pandas as pd

# Create sample data
data = {
   "Name": ["Arun", "Bala", "Charan", "Bala"],
   "Age": [21, 22, None, 22],
   "Marks": [85, 90, 78, 90]
}
df = pd.DataFrame(data)

# Save data as CSV and JSON
df.to_csv("input.csv", index=False)
df.to_json("input.json", orient="records", indent=4)

# Extract data from CSV
csv_data = pd.read_csv("input.csv")
```

```
print("Extracted CSV Data:")
print(csv_data)

# Transform data
csv_data = csv_data.drop_duplicates()
csv_data["Age"] = csv_data["Age"].fillna(
    csv_data["Age"].mean()
)

# Load cleaned data
csv_data.to_csv("output.csv", index=False)

# Extract data from JSON
json_data = pd.read_json("input.json")

print("\nExtracted JSON Data:")
print(json_data)

print("\nTransformed Data:")
print(csv_data)

print("\nETL Pipeline Completed Successfully!")
```

OUTPUT

Extracted CSV Data:

	Name	Age	Marks
0	Arun	21.0	85
1	Bala	22.0	90
2	Charan	NaN	78
3	Bala	22.0	90

Extracted JSON Data:

	Name	Age	Marks
0	Arun	21.0	85
1	Bala	22.0	90
2	Charan	NaN	78
3	Bala	22.0	90

Transformed Data:

	Name	Age	Marks
0	Arun	21.0	85
1	Bala	22.0	90
2	Charan	21.5	78

ETL Pipeline Completed Successfully!

INFERENCE:

The ETL pipeline demonstrated how Pandas can be used to extract data from CSV and JSON files, clean the data by removing duplicates, and handle missing values through transformation.

VIVA VOCE:

Q1. What is ETL?

Q2. What is Pandas?

Q3. What is the difference between CSV and JSON?

Q4. Which Pandas function is used to read a CSV file?

Q5. Which function is used to save a DataFrame as a CSV file?

RESULT

Thus, the ETL pipeline was successfully implemented using Python and Pandas to extract, transform, and load CSV/JSON data by removing duplicate records, handling missing values, and generating the cleaned output file.

Experiment No. 03	SQL for Analytics: Complex Queries with Window Functions, CTEs, and Optimization
Date	

AIM

To perform data analysis using SQL queries with Common Table Expressions (CTEs), window functions, and query optimization techniques to retrieve and analyze employee data efficiently.

ALGORITHM

1. Create an employee table with employee ID, name, department, and salary.
2. Insert sample employee records into the table.
3. Use a CTE to calculate the average salary of each department.
4. Apply the RANK() window function to rank employees based on salary within each department.
5. Retrieve employees earning more than their department's average salary.
6. Use appropriate indexes to improve query performance.
7. Display the query results.

PROGRAM / COMMAND

Step 1: Create Table

```
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   name VARCHAR(50),
   department VARCHAR(20),
   salary INT,
   experience INT
);
```

Step 2: Insert Data

```
INSERT INTO employees
(employee_id, name, department, salary, experience)
VALUES
(1, 'Anitha', 'HR', 35000, 2),
(2, 'Bala', 'IT', 50000, 4),
(3, 'Charan', 'HR', 40000, 3),
(4, 'Deepa', 'IT', 60000, 5),
```

(5, 'Ezhil', 'Finance', 45000, 3),

(6, 'Gokul', 'IT', 55000, 4),

(7, 'Harini', 'Finance', 48000, 5),

(8, 'Kiran', 'HR', 30000, 1);

Query 1: Display Employee Data

```
SELECT * FROM employees;
```

Output:

ID	Name	Department	Salary	Experience
1	Anitha	HR	35000	2
2	Bala	IT	50000	4
3	Charan	HR	40000	3
4	Deepa	IT	60000	5
5	Ezhil	Finance	45000	3
6	Gokul	IT	55000	4
7	Harini	Finance	48000	5
8	Kiran	HR	30000	1

Query 2: Overall Ranking Using a Window Function

```
SELECT
   name, department, salary,
   RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

Output:

Name	Department	Salary	Salary Rank
Deepa	IT	60000	1
Gokul	IT	55000	2
Bala	IT	50000	3
Harini	Finance	48000	4
Ezhil	Finance	45000	5
Charan	HR	40000	6

Anitha	HR	35000	7
Kiran	HR	30000	8

Query 3: Department-Wise Ranking

```
SELECT
   name,
   department,
   salary,
   RANK() OVER (
      PARTITION BY department
      ORDER BY salary DESC
   ) AS department_rank
FROM employees;
```

Output:

Name	Department	Salary	Department Rank
Charan	HR	40000	1
Anitha	HR	35000	2
Kiran	HR	30000	3
Harini	Finance	48000	1
Ezhil	Finance	45000	2
Deepa	IT	60000	1
Gokul	IT	55000	2
Bala	IT	50000	3

Query 4: CTE – Calculate Average Salary by Department

```
WITH department_salary AS (
   SELECT
      department,
      AVG(salary) AS average_salary
   FROM employees
   GROUP BY department
)
```

```
SELECT
   department,
   ROUND(average_salary, 2) AS average_salary
FROM department_salary
WHERE average_salary >= 40000;
```

Output:

Department	Average Salary
HR	35000.00 — excluded
Finance	46500.00
IT	55000.00

The actual query output contains only departments meeting the condition:

Department	Average Salary
Finance	46500.00
IT	55000.00

Query 5: Query Optimization Using an Index

Create an index on the department column.

```
CREATE INDEX idx_emp_department
ON employees(department);
```

Example query using the indexed column:

```
SELECT name, salary
FROM employees
WHERE department = 'IT';
```

Output:

Name	Salary
Bala	50000
Deepa	60000
Gokul	55000

INFERENCE

Window functions are used to rank employees based on salary, while the PARTITION BY clause provides department-wise rankings. CTEs simplify queries for calculating average salaries, and indexes can improve data retrieval performance.

VIVA VOCE

Q1. What is a window function?

Q2. What is RANK() in SQL?

Q3. What is a CTE?

Q4. What is PARTITION BY?

Q5. What is an index in SQL?

RESULT

Thus, SQL analytical queries using window functions, Common Table Expressions (CTEs), and indexing were implemented successfully to analyze employee salary data and improve query efficiency

Experiment No. 04	Build Batch Processing Pipeline Using PySpark for Large Dataset Analysis
Date	

AIM

To implement a batch processing pipeline using PySpark to process, transform, and analyze a large dataset efficiently using distributed data processing techniques.

ALGORITHM

1. Import the required PySpark libraries.
2. Create a Spark session.
3. Load the dataset into a Spark DataFrame.
4. Display the input data.
5. Remove duplicate records and handle missing values.
6. Group the data by category and calculate the total and average values.
7. Display the processed results.
8. Stop the Spark session.

PROGRAM

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import sum, avg


# Create Spark session

spark = SparkSession.builder \

    .appName("BatchProcessingPipeline") \

    .getOrCreate()


# Create sample dataset

data = [

    (1, "Laptop", "Electronics", 50000),

    (2, "Mobile", "Electronics", 20000),

    (3, "Chair", "Furniture", 5000),

    (4, "Table", "Furniture", 10000),

    (5, "Laptop", "Electronics", 50000)

]
```

```
columns = ["id", "product", "category", "price"]
df = spark.createDataFrame(data, columns)
print("Input Data:")
df.show()

# Remove duplicate records
cleaned_df = df.dropDuplicates()

# Handle missing values
cleaned_df = cleaned_df.dropna()

# Analyze data by category
result = cleaned_df.groupBy("category").agg(
    sum("price").alias("total_price"),
    avg("price").alias("average_price")
)
print("Batch Processing Result:")
result.orderBy("category").show()

# Stop Spark session
spark.stop()
```

OUTPUT

Input Data:

Id	Product	Category	Price
1	Laptop	Electronics	50000
2	Mobile	Furniture	20000
3	Chair	Furniture	5000
4	Table	Electronics	10000
5	Laptop	Electronics	50000

Batch Processing Result:

Category	Total_price	Average_price
Furniture	15000	7500.0
Electronics	70000	35000.0

INFERENCE

PySpark successfully processed the dataset by removing duplicate records, handling missing values, and performing category-wise aggregation using distributed data processing operations.

VIVA VOCE

Q1. What is PySpark?

Q2. What is batch processing?

Q3. What is a Spark DataFrame?

Q4. What is groupBy() used for?

Q5. Why is PySpark used for large datasets?

RESULT

Thus, a batch processing pipeline was successfully implemented using PySpark to clean, transform, and analyze dataset records and generate category-wise summary statistics.

Experiment No. 05	Create Data Warehouse Schema and Implement ETL to Load Data
Date	

AIM

To design a data warehouse schema and implement an ETL process using SQL to extract, transform, and load data into dimension and fact tables for analytical purposes.

ALGORITHM

1. Create a source table to store sales data.
2. Insert sample sales records into the source table.
3. Create dimension tables for customers and products.
4. Create a fact table to store sales transactions.
5. Extract records from the source table.
6. Transform the data by removing duplicates and organizing customer and product details.
7. Load the transformed data into the dimension and fact tables.
8. Execute a query to display the data warehouse results.

PROGRAM

Step 1: Create source table

```
CREATE TABLE Sales_Source (
   sale_id INT,
   customer_name VARCHAR(50),
   product_name VARCHAR(50),
   quantity INT,
   amount DECIMAL(10,2)
);
```

Step 2: Insert sample data

```
INSERT INTO Sales_Source VALUES
(1, 'Arun', 'Laptop', 2, 100000),
(2, 'Bala', 'Mobile', 3, 60000),
(3, 'Arun', 'Mobile', 1, 20000);
```

Step 3: Create customer dimension

```
CREATE TABLE Dim_Customer (
```

```
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50)
);
```

Step 4: Create product dimension

```
CREATE TABLE Dim_Product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50)
);
```

Step 5: Create sales fact table

```
CREATE TABLE Fact_Sales (
    sale_id INT PRIMARY KEY,
    customer_id INT,
    product_id INT,
    quantity INT,
    amount DECIMAL(10,2)
);
```

Step 6: Load customer dimension

```
INSERT INTO Dim_Customer
SELECT ROW_NUMBER() OVER (ORDER BY customer_name),
       customer_name
FROM (
    SELECT DISTINCT customer_name
    FROM Sales_Source
) AS C;
```

Step 7: Load product dimension

```
INSERT INTO Dim_Product
SELECT ROW_NUMBER() OVER (ORDER BY product_name),
       product_name
FROM (
    SELECT DISTINCT product_name
    FROM Sales_Source
) AS P;
```

Step 8: Load fact table

```
INSERT INTO Fact_Sales
SELECT S.sale_id, C.customer_id, P.product_id,
      S.quantity, S.amount
FROM Sales_Source S
JOIN Dim_Customer C
   ON S.customer_name = C.customer_name
JOIN Dim_Product P
   ON S.product_name = P.product_name;
```

Step 9: Display warehouse data

```
SELECT F.sale_id, C.customer_name,
      P.product_name, F.quantity, F.amount
FROM Fact_Sales F
JOIN Dim_Customer C
   ON F.customer_id = C.customer_id
JOIN Dim_Product P
   ON F.product_id = P.product_id
ORDER BY F.sale_id;
```

OUTPUT

sale_id	customer_name	product_name	quantity	amount
1	Arun	Laptop	2	100000.00
2	Bala	Mobile	3	60000.00
3	Arun	Mobile	1	20000.00

INFERENCE

The data warehouse schema organized customer and product information into dimension tables and sales transactions into a fact table. The ETL process extracted source records, transformed them by identifying unique customers and products, and loaded the data into the warehouse for analysis.

VIVA VOCE

Q1. What is a Data Warehouse?

Q2. What is a Star Schema?

Q3. What is a fact table?

Q4. What is a dimension table?

Q5. What are the three steps of ETL?

RESULT

Thus, a data warehouse schema was successfully created, and an ETL process was implemented using SQL to load sales data into dimension and fact tables for analytical reporting.

Experiment No. 06	Implement Real-Time Data Pipeline using Kafka for Streaming Data
Date	

AIM

To implement a real-time data pipeline using Apache Kafka to produce, stream, and consume messages for continuous data processing.

ALGORITHM

1. Install and configure Apache Kafka and Python.
2. Start the Kafka server and create a topic named data_stream.
3. Import the Kafka Python library.
4. Create a producer to send streaming messages to the Kafka topic.
5. Create a consumer to receive messages from the topic.
6. Display the received messages in real time.
7. Stop the producer and consumer after processing the messages.

PROGRAM

Create the producer program (producer.py) :

```
from kafka import KafkaProducer
import time
producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: v.encode("utf-8")
)
messages = [
    "Sensor Data: 25",
    "Sensor Data: 30",
    "Sensor Data: 35",
    "Sensor Data: 40"
]
for message in messages:
    producer.send("data_stream", message)
    print("Sent:", message)
    time.sleep(1)
```

```
producer.flush()
producer.close()
```

Create the consumer program (consumer.py) :

```
from kafka import KafkaConsumer
consumer = KafkaConsumer(
    "data_stream",
    bootstrap_servers="localhost:9092",
    auto_offset_reset="earliest",
    consumer_timeout_ms=5000,
    value_deserializer=lambda v: v.decode("utf-8")
)
for message in consumer:
    print("Received:", message.value)
consumer.close()
```

OUTPUT:

Producer Output

Sent: Sensor Data: 25

Sent: Sensor Data: 30

Sent: Sensor Data: 35

Sent: Sensor Data: 40

Consumer Output

Received: Sensor Data: 25

Received: Sensor Data: 30

Received: Sensor Data: 35

Received: Sensor Data: 40

INFERENCE

Apache Kafka enables real-time message streaming between producers and consumers. The producer sends sensor data to the Kafka topic, and the consumer receives and processes the messages continuously.

VIVA VOCE

Q1. What is Apache Kafka?

Q2. What is a Kafka topic?

Q3. What is a Kafka producer?

Q4. What is a Kafka consumer?

Q5. What is the difference between batch and real-time processing?

RESULT

Thus, a real-time data pipeline was implemented successfully using Apache Kafka and Python to produce, stream, and consume data messages.

Experiment No. 07	Orchestrate Data Pipelines using Apache Airflow
Date	

AIM

To create and execute a data pipeline using Apache Airflow to automate and schedule data processing tasks through a Directed Acyclic Graph (DAG).

ALGORITHM

1. Install Apache Airflow and its required dependencies.
2. Create a Python file to define the Airflow DAG.
3. Define the DAG with a unique ID and scheduling interval.
4. Create tasks to extract, transform, and load data.
5. Define task dependencies to specify the execution order.
6. Start the Airflow web server and scheduler.
7. Trigger the DAG and monitor task execution.
8. Verify the pipeline execution results.

PROGRAM

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator
# Define task functions
def extract():
    print("Extracting data...")
    return [10, 20, 30, 40]
def transform():
    data = [10, 20, 30, 40]
    result = [x * 2 for x in data]
    print("Transformed Data:", result)

def load():
    print("Data loaded successfully!")
# Define DAG
with DAG(
```

```
    dag_id="simple_etl_pipeline",
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False
) as dag:
    task1 = PythonOperator(
        task_id="extract_data",
        python_callable=extract
    )
    task2 = PythonOperator(
        task_id="transform_data",
        python_callable=transform
    )
    task3 = PythonOperator(
        task_id="load_data",
        python_callable=load
    )
    # Define task dependencies
    task1 >> task2 >> task3
```

OUTPUT

Terminal Output:

Extracting data...

Transformed Data: [20, 40, 60, 80]

Data loaded successfully!

Airflow DAG Execution:

simple_etl_pipeline

extract_data → transform_data → load_data

Status: Success

INFERENCE

Apache Airflow successfully orchestrates the ETL pipeline by managing task dependencies and executing extraction, transformation, and loading tasks in the specified order. The Airflow interface helps monitor task status and execution logs.

VIVA VOCE

Q1. What is Apache Airflow?

Q2. What is a DAG?

Q3. What is a task in Airflow?

Q4. What is a scheduler in Airflow?

Q5. How are task dependencies defined in Airflow?

RESULT

Thus, a data pipeline was successfully defined and orchestrated using Apache Airflow through a DAG with dependent tasks for extraction, transformation, and loading.

Experiment No. 08	Build Interactive Dashboard using Apache Superset for Business Insights
Date	

AIM

To create an interactive business dashboard using Apache Superset to visualize sales data, analyze business performance, and generate meaningful insights through charts and graphs.

ALGORITHM

1. Install and configure Apache Superset.
2. Start the Superset web server and initialize the database.
3. Create a sample sales dataset containing product, category, and sales information.
4. Connect the dataset to Apache Superset.
5. Create charts to analyze total sales and category-wise performance.
6. Add filters to enable interactive data exploration.
7. Arrange the charts into a dashboard.
8. Display the dashboard and analyze business insights.

PROGRAM

```
CREATE TABLE sales (
   sale_id INT PRIMARY KEY,
   product VARCHAR(50),
   category VARCHAR(50),
   sales_amount DECIMAL(10,2)
);


INSERT INTO sales VALUES
(1, 'Laptop', 'Electronics', 50000),
(2, 'Mobile', 'Electronics', 30000),
(3, 'Chair', 'Furniture', 10000),
(4, 'Table', 'Furniture', 15000),
(5, 'Headphones', 'Electronics', 5000);
```

OUTPUT

Sample Sales Data:

sale_id	product	category	sales_amount
1	Laptop	Electronics	50000
2	Mobile	Electronics	30000
3	Chair	Furniture	10000
4	Table	Furniture	15000
5	Headphones	Electronics	5000

Dashboard Results:

Business Insights Dashboard

Total Sales: ₹1,10,000

Category-wise Sales:

Electronics: ₹85,000

Furniture: ₹25,000

Highest Sales Category: Electronics

INFERENCE:

Apache Superset enables interactive visualization of sales data through KPI charts, bar charts, tables, and filters. The dashboard helps identify high-performing categories and supports data-driven business decisions.

VIVA VOCE

Q1. What is Apache Superset?

Q2. What is a dashboard?

Q3. What is data visualization?

Q4. What is the purpose of filters in a dashboard?

Q5. Name some tools used for dashboard creation.

RESULT

Thus, an interactive business dashboard was designed using Apache Superset to visualize sales data, analyze category-wise performance, and generate meaningful business insights.

.

Experiment No. 09	CONTAINERIZE DATA APPLICATION USING DOCKER
Date	

AIM

To containerize a Python data application using Docker by creating a Docker image and running the application inside an isolated container environment.

ALGORITHM

1. Install Docker Desktop and verify its installation.
2. Create a Python data application.
3. Create a Dockerfile to define the application environment.
4. Specify the base image, working directory, and required dependencies.
5. Build a Docker image using the Dockerfile.
6. Run the application inside a Docker container.
7. Verify the application output.
8. Stop the container after execution.

PROGRAM

```
#app.py
print("Data Application Started")

data = [10, 20, 30, 40, 50]

total = sum(data)
average = total / len(data)

print("Dataset:", data)
print("Total:", total)
print("Average:", average)
print("Application executed successfully!")

#Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY app.py .

CMD ["python", "app.py"]
```

OUTPUT

Docker Image Build Output:

Successfully built <image-id>

Successfully tagged data-app:latest

Application Output:

Data Application Started

Dataset: [10, 20, 30, 40, 50]

Total: 150

Average: 30.0

Application executed successfully!

INFERENCE

Docker successfully packages the Python application and its runtime environment into a portable image. The container executes the application in an isolated environment, ensuring consistent execution across systems with Docker installed.

VIVA VOCE

Q1. What is Docker?

Q2. What is a Docker image?

Q3. What is a Docker container?

Q4. What is a Dockerfile?

Q5. What is the purpose of containerization in data engineering?

RESULT

Thus, the Python data application was successfully containerized using Docker, built into an image, and executed inside a Docker container.

Experiment No. 10	END-TO-END DATA PIPELINE: INGESTION TO VISUALIZATION
Date	

AIM

To develop an end-to-end data pipeline using Python and Pandas to ingest raw data, clean and transform it, store the processed data in a CSV file, and visualize the results using charts.

ALGORITHM

1. Import the required Python libraries.
2. Create a sample dataset containing product sales information.
3. Ingest the data into a Pandas DataFrame.
4. Clean the data by removing duplicate records and handling missing values.
5. Transform the data by calculating total sales for each product.
6. Store the processed data in a CSV file.
7. Visualize the results using a bar chart.
8. Display the processed data and visualization.

PROGRAM

```
import pandas as pd
import matplotlib.pyplot as plt
# Step 1: Data ingestion
data = {"Product": ["Laptop", "Mobile", "Tablet",
            "Laptop", "Mobile"],
   "Quantity": [2, 3, 1, 2, 1],
   "Price": [50000, 20000, 15000, 50000, 20000] }
df = pd.DataFrame(data)
print("Raw Data:")
print(df)
# Step 2: Data cleaning
df = df.drop_duplicates()
df = df.dropna()
# Step 3: Data transformation
df["Total_Sales"] = df["Quantity"] * df["Price"]
# Step 4: Data storage
```

```
df.to_csv("processed_data.csv", index=False)
print("\nProcessed Data:")
print(df)
# Step 5: Data aggregation
sales = df.groupby("Product")["Total_Sales"].sum()
print("\nProduct-wise Sales:")
print(sales)
# Step 6: Data visualization
sales.plot(kind="bar", title="Product-wise Sales")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.savefig("sales_chart.png")
plt.show()
print("\nPipeline Executed Successfully!")
```

OUTPUT

Raw Data :

	Product	Quantity	Price
0	Laptop	2	50000
1	Mobile	3	20000
2	Tablet	1	15000
3	Laptop	2	50000
4	Mobile	1	20000

Processed Data :

	Product	Quantity	Price	Total_Sales
0	Laptop	2	50000	100000
1	Mobile	3	20000	60000
2	Tablet	1	15000	15000

Product-wise Sales :

```
Product
Laptop    100000
Mobile     60000
Tablet     15000
```

INFERENCE

The end-to-end data pipeline successfully ingested raw sales data, removed duplicate records, calculated total sales, stored the processed data in a CSV file, and visualized product-wise sales using a bar chart.

VIVA VOCE

Q1. What is an end-to-end data pipeline?

Q2. What are the major stages of a data pipeline?

Q3. Why is data cleaning important?

Q4. Why is visualization used in a data pipeline?

Q5. What is the difference between data processing and data visualization?

RESULT

Thus, an end-to-end data pipeline from data ingestion to visualization was successfully implemented using Python, Pandas, and Matplotlib.